CS3230 : Design and Analysis of Algorithms (Spring 2014)
Tutorial Set #3
[SOLUTION SKETCHES with Suggestions]

R1. (Exercise 7.1-1 on p. 171 of [CLRS]) [Understanding PARTITION in Quicksort]
Using Figure 7.1 as a model, illustrate the operation of PARTITION on the array
$A = [13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]$.
(Use the *last element* as the pivot element. Of course, you swap it to the front *first*.)

Solution sketch:

$A = [13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, \mathbf{11}]$.	(select pivot 11)
$A = [\mathbf{11}, \mid\mid\mid \mathbf{19}, 9, 5, 12, 8, 7, 4, 21, 2, 6, \mathbf{13}]$.	(swap pivot to front)
$A = [\mathbf{11}, \mid\mid \mathbf{19}, \mid \mathbf{9}, 5, 12, 8, 7, 4, 21, 2, 6, 13]$.	(19 is big)
$A = [\mathbf{11}, \mid 9, \mid \mathbf{19}, \mid \mathbf{5}, 12, 8, 7, 4, 21, 2, 6, 13]$.	(9 small, swap)
$A = [\mathbf{11}, \mid 9, 5, \mid 19, \mid \mathbf{12}, 8, 7, 4, 21, 2, 6, 13]$.	(5 is small, swap)
$A = [\mathbf{11}, \mid 9, 5, \mid \mathbf{19}, 12, \mid \mathbf{8}, 7, 4, 21, 2, 6, 13]$.	(12 is big)
$A = [\mathbf{11}, \mid 9, 5, 8, \mid 12, 19, \mid \mathbf{7}, 4, 21, 2, 6, 13]$.	(8 is small, swap)
$A = [\mathbf{11}, \mid 9, 5, 8, 7, \mid \mathbf{19}, 12, \mid \mathbf{4}, 21, 2, 6, 13]$.	(7 is small, swap)
$A = [\mathbf{11}, \mid 9, 5, 8, 7, 4, \mid 12, 19, \mid \mathbf{21}, 2, 6, 13]$.	(4 is small, swap)
$A = [\mathbf{11}, \mid 9, 5, 8, 7, 4, \mid \mathbf{12}, 19, 21, \mid \mathbf{2}, 6, 13]$.	(21 is big)
$A = [\mathbf{11}, \mid 9, 5, 8, 7, 4, 2, \mid \mathbf{19}, 21, 12, \mid \mathbf{6}, 13]$.	(2 is small, swap)
$A = [\mathbf{11}, \mid 9, 5, 8, 7, 4, 2, 6, \mid 21, 12, 19, \mid \mathbf{13}]$.	(6 is small, swap)
$A = [\mathbf{11}, \mid 9, 5, 8, 7, 4, 2, 6, \mid 21, 12, 19, 13 \mid]$.	(13 is big)
$A = [\mathbf{6}, 9, 5, 8, 7, 4, 2, \mid \mathbf{11}, \mid 21, 12, 19, 13 \mid]$.	(swap pivot to middle)

R2. [Analysis of Randomized Quicksort: (n+1) or (n) or (n–1), does it matter?**]**
In the detailed analysis of Randomized Quicksort given in the lecture notes (slides 35-43), we used the recurrence $T(n) = T(k–1) + T(n–k) + (n+1)$, where the term ($n$+1) was the time taken to partition the array $A[1..n]$.

(a) Suppose that we insist that the term in the recurrence should be (n–1) instead, since we do (n–1) comparisons in the partitioning subroutine given in the lecture notes (slide 4). Will this affect the result for $T(n) = 1.386\, n \lg n + O(n)$ (slide 43)?
Redo the entire analysis (slides 35-43) with (n–1) instead of (n+1). What is the difference?

$$nT(n) = 2\sum_{k=0}^{n-1} T(k) + n(n-1)$$ (You should get THIS at some point.)

Then, we first get rid of "full history".

$$nT(n) = 2\left[T(0) + T(1) + \ldots + T(n-2) + T(n-1)\right] + n(n-1)$$

$$(n-1)T(n-1) = 2\left[T(0) + T(1) + \ldots + T(n-2)\right] + (n-1)(n-2)$$

$$nT(n) = (n+1)T(n-1) + 2n - 2$$

Then turn it into something that *telescopes*

$$nT(n) = (n+1)T(n-1) + 2n - 2$$

$$\frac{T(n)}{(n+1)} = [\frac{2}{(n+1)} - \frac{2}{n(n+1)}] + \frac{T(n-1)}{(n)}$$ (Divide by $n(n+1)$.)

Then, at this point, you can either telescope or you can also "*carefully look at the math*".

If you have "*carefully looked at the math*", but still cannot get it,
then, you do the actual telescoping, and the summing, and you will get this

$$\begin{aligned}
\frac{T(n)}{(n+1)} &= (\frac{2}{n+1} - \frac{2}{n(n+1)}) + \frac{T(n-1)}{(n)} \\
&= (\frac{2}{n+1} - \frac{2}{n(n+1)}) + (\frac{2}{n} - \frac{2}{(n-1)(n)}) + \frac{T(n-2)}{(n-1)} \\
&= (\frac{2}{n+1} - \frac{2}{n(n+1)}) + (\frac{2}{n} - \frac{2}{(n-1)(n)}) + (\frac{2}{n-1} - \frac{2}{(n-2)(n-1)}) + \frac{T(n-3)}{(n-2)} \\
&= (\frac{2}{n+1} - \frac{2}{n(n+1)}) + (\frac{2}{n} - \frac{2}{(n-1)(n)}) + (\frac{2}{n-1} - \frac{2}{(n-2)(n-1)}) + \ldots + (\frac{2}{3} - \frac{2}{2.3}) + \frac{T(1)}{2} \\
&= 2(\frac{1}{n+1} + \frac{1}{n} + \frac{1}{n-1} + \ldots + \frac{1}{3}) - 2(\frac{1}{n(n+1)} + \frac{1}{(n-1)n} + \frac{1}{(n-2)(n-1)} + \ldots + \frac{1}{2.3}) + O(1)
\end{aligned}$$

This first part will give you the 1.386 n lg n + $O(n)$. (same as original analysis)
The second part will sum to $O(n)$. Your DIY-HW. (this is the "difference")
Hence, it will not affect the 1.386 n lg n dominant term in the running time.
Running time is still 1.386 n lg n

(b) If you were to repeat **(a)** but now with the term (n+1) replaced by (n), what do you *think* is the difference. Answer this *without* redoing the entire analysis.

Answer: This time, you definitely need to "*carefully look at the math*".

R3. [Closest Pair with Divide and Conquer]
You are given an unsorted array $A[1..n]$ of numbers. You want to find the *closest pair*, namely two elements $A[i]$ and $A[j]$, (with $i \neq j$) for which $|A[i] - A[j]|$ is the *minimum* (among all possible pairs). Give a *divide-and-conquer* algorithm for solving the closest pair problem in $O(n \lg n)$ worst-case time.

[**Note:** There are, of course, other strategies to solve this as well, but this problem asks for a *divide-and-conquer* algorithms.]

Answer: DIY.

R4. [Merging Multiple Lists Efficiently]
You are given k sorted arrays, with n elements each, and you want to suitably combine them into a single sorted array (with kn elements).

(a) One strategy to do this is to use a 2-way merge to merge the first two sorted arrays, then merge the result with the third sorted array, then merge in the fourth, and so on. What is the time complexity of this algorithm, in terms of k and n?
Answer: Iterative merge algorithm: $\Theta(nk^2)$

(b) Design a more *efficient* algorithm for solving this problem. What is the time complexity of your algorithm, in terms of k and n?
Answer: Divide-and-Conquer algorithm: $\Theta(n(k \lg k))$

D1. [Re-looking at Programming Assignment – Contributed by Steven Halim, 2014]

(a) Present your solutions (*not* your code) for Progr-Asg 1, Subtasks A/B/C. Do not give code, but describe the entities you store, and data structures & algorithms used in your solution.

(b) Analyze the time complexity of your solution in terms of N and Q.
(**Note:** Recall that N is the *maximum* number of students in the data structure at any time, and Q is the *maximum* number of queries (namely, lines with $CMD = 3$ or $CMD = 4$).

(c) Also analyze the "code complexity" (or "implementation complexity") of your solution. Here, "code complexity" or "implementation complexity" is *loosely defined* to mean "how much effort that you need to implement that algorithm".
(There is no need for you to use more precise definitions of "code complexity" typically seen in SE courses, such as *code complexity metrics*.)

Answer and Remarks by Steven Halim:

This problem is the standard "Order Statistics" problem as outlined in [CLRS]-Ch9,14 *and* [CP3]-Section 9.29. *Prof Leong mentioned the required (GENERAL/Classic) solution for this problem, but is that the easiest or most appropriate solution for ALL constraints?*

Abridged problem description: Given N students that are uniquely identified by their integer scores ([0..1B]), *answer Q queries: Select*(V) *and Rank*(V)
In general form, the data can be dynamic, new student appears or existing students deleted, but not all subtasks has this property.

Subtask 1/A: $1 \le N \le 100$, $1 \le Q \le 100$, *only Select(V) is asked.* (*LHW-Comment: Problem is small.*)
For this, $O(QN^2)$ *brute force solution is OK, but actually harder to implement.*
We can use the C++ library solution n^{th}*_element or write our own O(N) RandomizedSelect as taught in CLRS to handle Select*(V).
The easiest solution is therefore to sort the array first and then report the content of arr[V-1].
Since there are several deletions, we need to keep re-sorting the data after each deletion.
(LHW-Question: In this case, what sorting algorithm is ideal for this re-sorting?)

Subtask 2/B: $1 \le N \le 100$, $1 \le Q \le 100$, *only Rank(V) is asked.* (*LHW-C: also small problem.*)
For rank (Subtask 2/B), I do not see any other easy way other than sorting the array first, and then do binary search.
Since there are several deletions, we need to keep re-sorting the data after each deletion.

Subtask 3/C: $1 \le N \le 10000$, $1 \le Q \le 100000$, *with both Select(V) and Rank(V), but it is guaranteed that all Select(V)/Rank(V) queries appear ONLY AFTER the last add operation (and no delete operation).*

This is a very relaxing (simplifying) assumption of the input data.
Therefore, after the last add operation is processed, we just need to sort the array ONCE in descending order, this is O(N log N).
For Select(V), we simply return arr[V-1] if V <= N, or -1 otherwise. This is O(1) per CMD=3.
For Rank(V), we simply binary search arr to find the occurrence "r" of V, and report r+1, or -2 if v is not found. This is O(log N) per CMD=4.
Overall: $O(N \log N + Q * (1 + \log N)) = O((N+Q)\lg N)$, *0.041s in C++, should be OK in Java.*
Trying to sort the array multiple times (*using whatever strategy: after each insertion/deletion or before select/rank query*) *is TLE, > 2s.*

Common mistakes:
1. Sort MANY times for Subtask 3/C --> TLE

2. Accidentally use O(n) method Contains and/or indexOf functions for Subtask 3/C --> TLE
From: http://docs.oracle.com/javase/7/docs/api/java/util/ArrayList.html, we have:
"The size, isEmpty, get, set, iterator, and listIterator operations run in constant time. The add operation runs in amortized constant time, that is, adding n elements requires O(n) time. ALL OF THE OTHER OPERATIONS RUN IN LINEAR TIME (ROUGHLY SPEAKING). The constant factor is low compared to that for the LinkedList implementation."

D2. [Weighing Problem – Contributed by Tan Lee Eng, 1991]

You are given 27 coconuts labeled 1, 2, . . . , 27, and *only* a beam balance. Among these 27 coconuts, 26 of them have exactly the *same* weight, but one of them is *lighter*. We want an algorithm to identify the lighter coconut using *only* the beam balance (which gives three possible outcomes: left-side is heavy, both-sides balanced, right-side is heavy).

(a) Before solving this, how about solving the smaller 3 coconuts problem. Give a simple algorithm for doing this. Specify which coconuts to put onto the beam balance and what to do for each possible outcome of the weighing.

Solution: from LHW

Here the possible outcomes are 1L, 2L, 3L, assuming coconuts are labeled 1,2,3.

Compare (1:2) using the beam balance.

Outcome: Balanced ➔ 3L (coconut 3 is light)

Outcome: Left-side heavy ➔ 2L (coconut 2 is light)

Outcome: Right-side heavy ➔ 1L (coconut 1 is light)

Thus, 1 weighing is sufficient.

(b) Now, give an algorithm to solve the original 27 coconuts problem, namely to identify the lighter coconut using the least number of weighing with the beam balance.

Nice solution from Steven:

We generalize this process by splitting 27 coconuts into 9 (set G1), 9 (set G2), 9 (set G3), and do similar process as in part (a) above. (Think of coconut 1 as G1, 2 as G2, 3 as G3)

Then, in one weighing step, we can shrink the problem size from 27 to 9.

(Here 1L will mean that the lighter coconut is in the set G1, and so on.)

(LHW-Comment: Can you see concept of "3-*way balance*" at work in this problem?)

Then, in two weighing steps, we go down to 3 (27 -> 9 -> 3.)

Finally, in three weighing steps, we are have (27 -> 9 -> 3 -> 1), done :).

(c) [**Question:** What, if anything, does this problem have to do with Analysis of Algorithms? Or, why is Prof. Leong asking this weird question in CS3230?] (Short answers please.)

[Comments by LHW]

Yes, of course. The problem has lots to do with analysis of algorithms. Specifically on proving lower bound for binary search, for sorting, or more general, on decision-tree based algorithms (that includes many AI-type algorithms).

In a successful binary search for a number x in a sorted array $A[1..n]$, there are n possible outcomes. If we probe (make a comparison) at $A[7]$, then there are also three possibilities: found x at A[7], x is smaller (meaning it is one the left side, meaning 6 possible outcomes) or x is larger (meaning it is one the right side, meaning $(n–7)$ possible outcomes). For large n, this is *unbalanced* and leads to bad worst case time. Which is why, in binary search, we always probe in the middle, so that in either case, there are $n/2$ possible outcomes.

For comparison-based sorting, we look at the "decision tree" and whenever we compare $A[i]:A[j]$, we decompose the possible outcomes ("permutations") into those where $A[i]<A[j]$ in the left subtree, and those where $A[i]>A[j]$ in the right subtree. Again, we seek *balance* in the possible outcomes in both subtrees. (See section on "Lower Bounds for Comparison-Based Sorting", [CLRS]-Ch8.1 for more in-depth discussion.)

D3. [Twenty-Something Quicksort, contributed by Ken Sung, 2013]
A CS3230 student, called Twenty-Something (who is also 20-something years old) said that she has an algorithm that, given an unsorted array $A[1..n]$, can find a "20-something pivot" in time $P(n) = 6n + o(n)$. Using her magic "20-something *pivot*", she can guarantee that *each* of the two partitions has at least between 20% to 30% of the n elements.

Let $TS(n)$ be the *worst-case performance* of Ms. Twenty-Something's Quicksort. Give a recurrence relation for $TS(n)$. Solve for $TS(n)$ and give it in Θ-notations.
[**Note:** In your analysis, you can safely "throw away" the $o(n)$ factor in $P(n)$.]

Modified from solution from Steven Halim:
If we assume a 20% partition (higher cost), then
$TS(n) = TS(n/5) + TS(4n/5) + 6n$ (cannot use Master Theorem, ignore $o(n)$)
Draw recursion tree (but can "***re-use the method***" recursion tree)
Each level actually sums up to $6n$.
Using the longer (right) branch: $\log_{5/4}(n)$

$$TS(n) \le 6n\log_{5/4}(n) = 6n(\tfrac{\ln 2}{\ln(5/4)}\lg n) = 6*3.106(n\lg n) = 18.63n\lg n$$

Using the shorter (left) branch: $\log_5(n)$

$$TS(n) \ge 6n\log_5(n) = 6n(\tfrac{\ln 2}{\ln 5}\lg n) = 6*0.431(n\lg n) = 2.584n\lg n$$

Hence $TS(n) = \Theta(n \lg n)$.

Note that if you had used a 30% partition, then the lower bound is bigger, while the upper bound is smaller, because the 30% partition does not give a worst-case performance.

As an exercise, you can carry out this analysis for the 30% partition and check out the lower, upper bounds yourself.